Rapport de Séance 2 — NoSQL Security

Modélisation, Requêtes, Pipeline, Index & Sécurité Data-Level

2e année Cybersécurité | MongoDB 7.0.31 | Docker | mongosh 2.8.1

Base: soc_lite | 2 avril 2026

Table des matières

1 Modélisation NoSQL : Embed vs Reference **3**
1.1 Création des 3 Collections . . . 3
1.1.1 Users — Rôles embedded (One-to-Few) . . . 3
1.1.2 Security_logs — Logs référencés (One-to-Many) . . . 4
1.1.3 Incidents — Hybride (Embed comments + Reference logs) . . . 4
1.2 Vérification & Auto-check . . . 5

2 Patterns de Relations & Manipulation **6**
2.1 TP 2 — $push, Reference & Many-to-Many . . . 6
2.1.1 Ajouter un commentaire (Embed : $push) . . . 6
2.1.2 3 nouveaux logs + Incident INC-2025-003 (Reference) . . . 6
2.1.3 Many-to-Many : $addToSet . . . 7

3 Requêtes SOC Avancées **8**
3.1 R1–R6 : Requêtes guidées . . . 8
3.1.1 R1 — Logs sévérité critical (3 docs) . . . 8
3.1.2 R2 — Logs 10h–12h, tri croissant (5 docs) . . . 8
3.1.3 R3 — login_failed avec projection minimale (6 docs) . . . 8
3.1.4 R4 — 3 derniers logs . . . 9
3.1.5 R5 — $or : IP OU critical (6 docs) . . . 9
3.1.6 R6 — Incidents actifs + commentaires (2 docs) . . . 9
3.2 E1–E4 : Exercices autonomes . . . 10

4 Aggregation Pipeline **11**
4.1 A1 — Top 5 IPs sources . . . 11
4.2 A2 — Échecs login par utilisateur . . . 11
4.3 A3 — Répartition des sévérités . . . 12
4.4 A4 — Alertes par device . . . 12
4.5 A5 — Événements par heure . . . 12

5 Index & explain() **14**
5.1 Étape 1 — Sans index (COLLSCAN) . . . 14
5.2 Étape 2 & 3 — Index simple (IXSCAN) . . . 14
5.3 Étape 4 — Index composé . . . 15
5.4 Prefix rule — Piège . . . 15

6 Sécurité Data-Level **17**
6.1 TP 6 — PII & Projections sûres 17
6.1.1 Masquer les emails 17
6.1.2 Projection sûre (viewer) & Exclusion raw_log 18
6.2 Checklist Sécurité Data — SOC-lite 18

Chapitre 1

Modélisation NoSQL : Embed vs Reference

En SQL, on normalise avec des clés étrangères et jointures. En NoSQL orienté document, il n'y a pas de jointures natives efficaces. Pour chaque relation, il faut décider si l'on **imbrique** (embed) les données dans un seul document, ou si on les **sépare** dans des collections liées par un identifiant (reference).

Ce choix conditionne la performance, la taille des documents, la cohérence des mises à jour, et — crucial en sécurité — la **surface d'attaque**. Un document trop chargé exposerait tout lors d'une requête mal protégée.

Règle de choix

Données bornées + lues ensemble → **Embed** Données illimitées / indépendantes → **Reference** Données sensibles à isoler → **Reference**

Erreur fréquente

Imbriquer 50 000 logs dans un incident ferait exploser la limite de 16 Mo, ralentirait les lectures, et exposerait les logs bruts à quiconque lit l'incident — violation du principe de moindre privilège au niveau données.

1.1 Création des 3 Collections

1.1.1 Users — Rôles embedded (One-to-Few)

Les rôles sont peu nombreux (2–3), toujours lus avec l'utilisateur, rarement modifiés → **Embed**.

```
\prompt db.users.insertMany([
...   { username: "alice_analyst", full_name: "Alice Dupont", email: "
   alice@soc.local",
...     roles: ["analyst", "viewer"], team: "Blue Team", is_active:
   true,
...     created_at: new Date("2025-01-15"), last_login: new Date
   ("2025-03-01T08:30:00Z") },
```

```
...   { username: "bob_senior", full_name: "Bob Martin", email: "
   bob@soc.local",
...     roles: ["senior_analyst", "incident_manager"], team: "Blue
   Team", is_active: true,
...     created_at: new Date("2024-06-10"), last_login: new Date
   ("2025-03-03T14:12:00Z") },
...   // ... charlie_admin, diana_junior (même structure)
... ])
```

```
{ acknowledged: true, insertedIds: { '0': ObjectId('...d0'), '1':
   ObjectId('...d1'),
  '2': ObjectId('...d2'), '3': ObjectId('...d3') } }
```

1.1.2 Security_logs — Logs référencés (One-to-Many)

Un incident peut impliquer des milliers de logs. Les imbriquer dépasserait 16 Mo et empêcherait de les requêter indépendamment → **Reference**.

```
\prompt db.security_logs.insertMany([
...   { timestamp: new Date("2025-03-03T10:05:22Z"),
...     event_type: "login_failed", src_ip: "192.168.1.105", dst_ip:
   "10.0.0.2",
...     src_port: 44821, dst_port: 22, protocol: "SSH", user_attempted
   : "root",
...     device_id: "srv-web-01", severity: "medium",
...     raw_log: "Failed password for root from 192.168.1.105 port
   44821 ssh2" },
...   // ... 7 autres documents (login_success, port_scan,
   malware_detected,
...   //     data_exfiltration_suspect, etc.)
... ])
```

```
{ acknowledged: true, insertedIds: { '0': ObjectId('...d4') ... '7':
   ObjectId('...db') } }
```

1.1.3 Incidents — Hybride (Embed comments + Reference logs)

Les `comments` sont embedded (borné < 30, toujours lus avec l'incident). Les `related_log_ids` sont référencés (illimité, volumineux, indépendants).

```
\prompt let logIds = db.security_logs.find({ src_ip: "192.168.1.105"
   }).map(d => d._id)
\prompt db.incidents.insertMany([
...   { incident_id: "INC-2025-001",
...     title: "Brute force SSH sur srv-web-01", severity: "high",
   status: "investigating",
...     assigned_to: "alice_analyst", related_log_ids: logIds,
...     comments: [
...       { author: "alice_analyst",
...         text: "3 tentatives en 2s. Pattern brute-force confirmé.",
...         timestamp: new Date("2025-03-03T10:35:00Z") },
...       { author: "bob_senior",
...         text: "IP bloquée au firewall.",
...         timestamp: new Date("2025-03-03T10:50:00Z") }
```

```
...     ],
...     mitre_tactics: ["TA0006"], mitre_techniques: ["T1110.001"] },
...   // ... INC-2025-002 (malware + exfiltration)
... ])
```

1.2 Vérification & Auto-check

```
\prompt db.users.countDocuments()          //     4
\prompt db.security_logs.countDocuments()  //     8
\prompt db.incidents.countDocuments()      //     2
```

Q : Pourquoi les logs sont-ils référencés et non imbriqués ?

R : Un incident peut impliquer des milliers de logs. Les imbriquer dépasserait la limite de 16 Mo et empêcherait de requêter les logs indépendamment.

Checkpoint 1 — Risque principal d'un document dépassant 16 Mo ?

C : MongoDB refuse l'insertion/mise à jour. Le document est rejeté.

Chapitre 2

Patterns de Relations & Manipulation

Les 5 patterns : One-to-Few (embed), One-to-Many (reference), Many-to-Many (tableaux bilatéraux), Time-series (append-only), Dénormalisation contrôlée.

Sécurité — La duplication multiplie les surfaces

Si le nom complet d'un analyste est dupliqué dans 50 000 logs, une fuite sur `security_logs` expose aussi des données personnelles. **Règle :** ne dupliquer que des données non sensibles et rarement modifiées (un username est acceptable, un téléphone ne l'est pas).

2.1 TP 2 — $push, Reference & Many-to-Many

2.1.1 Ajouter un commentaire (Embed : $push)

```
\prompt db.incidents.updateOne(
...   { incident_id: "INC-2025-001" },
...   { $push: { comments: {
...     author: "diana_junior",
...     text: "L'IP 192.168.1.105 = sous-réseau visiteur.",
...     timestamp: new Date()
...   }},
...   $set: { updated_at: new Date() }
... )
```

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

2.1.2 3 nouveaux logs + Incident INC-2025-003 (Reference)

Trois nouveaux logs pour charlie (dont une **privilege_escalation** critique à 17 :01), puis création d'un incident lié par ObjectId :

```
\prompt let charlieLogIds = db.security_logs.find({
...   src_ip: "10.0.1.200",
...   timestamp: { $gte: new Date("2025-03-03T17:00:00Z") }
... }).map(d => d._id)
```

```
\prompt db.incidents.insertOne({
...   incident_id: "INC-2025-003",
...   title: "Tentative d'escalade de privilèges - charlie",
...   severity: "critical", status: "new",
...   assigned_to: "alice_analyst",
...   related_log_ids: charlieLogIds, comments: [],
...   mitre_tactics: ["TA0004"], mitre_techniques: ["T1078"]
... })
```

```
\prompt db.security_logs.countDocuments()
```

```
11
```

2.1.3 Many-to-Many : $addToSet

```
\prompt db.incidents.updateOne(
...   { incident_id: "INC-2025-002" },
...   { $addToSet: { assigned_analysts: { $each: ["bob_senior","
   alice_analyst"] } } }
... )
```

Q : Si on supprime un log, que se passe-t-il dans l'incident ?

R : Rien automatiquement. L'ObjectId reste dans `related_log_ids` → référence cassée. À gérer au niveau applicatif.

Chapitre 3

Requêtes SOC Avancées

Trois réflexes pour chaque requête SOC : **1)** Filtrer au maximum côté serveur. **2)** Projeter uniquement les champs nécessaires. **3)** Limiter les résultats (`.limit()`).

Oublier la projection

`db.users.find({team: "Blue Team"})` sans projection retourne tous les champs, y compris email. Si cette requête alimente un dashboard public, des données personnelles sont exposées (RGPD, art. 5).

3.1 R1–R6 : Requêtes guidées

3.1.1 R1 — Logs sévérité critical (3 docs)

```
\prompt db.security_logs.find({ severity: "critical" })
```

```
[
  { event_type: 'malware_detected',          src_ip: '192.168.2.30'
     },
  { event_type: 'data_exfiltration_suspect',  src_ip: '10.0.0.10'
     },
  { event_type: 'privilege_escalation',      src_ip: '10.0.1.200'    }
]
```

3.1.2 R2 — Logs 10h–12h, tri croissant (5 docs)

```
\prompt db.security_logs.find({
...   timestamp: {
...     $gte: new Date("2025-03-03T10:00:00Z"),
...     $lte: new Date("2025-03-03T12:00:00Z")
...   }
... }).sort({ timestamp: 1 })
```

3.1.3 R3 — login_failed avec projection minimale (6 docs)

```
\prompt db.security_logs.find(
...   { event_type: "login_failed" },
...   { src_ip: 1, user_attempted: 1, timestamp: 1, _id: 0 }
... )
```

```
[
  { timestamp: 10:05:22, src_ip: '192.168.1.105', user_attempted: '
      root'    },
  { timestamp: 10:05:23, src_ip: '192.168.1.105', user_attempted: '
      root'    },
  { timestamp: 15:00:00, src_ip: '10.0.1.200',    user_attempted: '
      charlie' },
  // ... 3 autres
]
// raw_log, device_id, ports EXCLUS
```

3.1.4 R4 — 3 derniers logs

```
\prompt db.security_logs.find().sort({ timestamp: -1 }).limit(3)
```

```
[
  { event_type: 'privilege_escalation', timestamp: 17:01:00, severity:
      'critical' },
  { event_type: 'login_failed',        timestamp: 17:00:05, severity:
      'medium'   },
  { event_type: 'login_failed',        timestamp: 17:00:00, severity:
      'medium'   }
]
```

3.1.5 R5 — $or : IP OU critical (6 docs)

```
\prompt db.security_logs.find({
...   $or: [ { src_ip: "192.168.1.105" }, { severity: "critical" } ]
... })
```

3.1.6 R6 — Incidents actifs + commentaires (2 docs)

```
\prompt db.incidents.find(
...   { status: { $in: ["active","investigating"] } },
...   { incident_id: 1, title: 1, "comments.author": 1, "comments.text
    ": 1, _id: 0 }
... )
```

```
[
  { incident_id: 'INC-2025-001', title: 'Brute force SSH sur srv-web
      -01',
    comments: [
      { author: 'alice_analyst',  text: '3 tentatives en 2s. Pattern
          confirmé.' },
      { author: 'bob_senior',     text: 'IP bloquée au firewall.' },
```

```
      { author: 'diana_junior',  text: "L'IP = sous-réseau visiteur."
         }
    ] },
  { incident_id: 'INC-2025-002', title: 'Malware détecté sur endpoint-
     pc-042',
    comments: [
      { author: 'bob_senior', text: 'Isolement réseau effectué.
         Forensique en cours.' }
    ] }
]
```

3.2 E1–E4 : Exercices autonomes

```
// E1 - Blue Team actifs (3 docs)
\prompt db.users.find(
...    { team: "Blue Team", is_active: true },
...    { username: 1, roles: 1, _id: 0 }
... )
```

```
// E2 - siem-console, 5 derniers
\prompt db.security_logs.find({ device_id: "siem-console" })
...    .sort({ timestamp: -1 }).limit(5)
```

```
// E3 - Compter login_failed
\prompt db.security_logs.countDocuments({ event_type: "login_failed"
   }) //     6
// E4 - Incidents d'alice
\prompt db.incidents.find(
...    { assigned_to: "alice_analyst" },
...    { title: 1, severity: 1, _id: 0 }
... )
```

Checkpoint 2

Q : Différence `$push` vs `$addToSet` ?
C : `$push` ajoute même si doublon. `$addToSet` n'ajoute que si absent.

Chapitre 4

Aggregation Pipeline

`find()` filtre et projette, mais ne calcule pas de statistiques. L'**aggregation pipeline** transforme les données par étapes, comme un pipeline Unix. Les 4 stages essentiels pour un SOC :

- `$match` — filtrer (placer **en premier** pour exploiter les index)
- `$group` — regrouper par clé avec accumulateurs (`$sum`, `$max`, `$addToSet`)
- `$sort` — trier (1 = asc, –1 = desc)
- `$limit` — tronquer

Sécurité — L'aggregation comme outil de détection

Regrouper les échecs par IP identifie le brute-force. Calculer le volume sortant par destination détecte l'exfiltration. Ces pipelines sont le fondement des règles d'alerte SIEM.

4.1 A1 — Top 5 IPs sources

```
\prompt db.security_logs.aggregate([
...   { $group: { _id: "$src_ip", count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 5 }
... ])
```

```
[
  { _id: '10.0.1.200',     count: 4 },  // Attaquant interne principal
  { _id: '192.168.1.105', count: 3 },  // Brute-force externe
  { _id: '10.0.1.50',      count: 1 },
  { _id: '172.16.0.88',    count: 1 },
  { _id: '10.0.0.10',      count: 1 }
]
```

4.2 A2 — Échecs login par utilisateur

```
\prompt db.security_logs.aggregate([
...   { $match: { event_type: "login_failed" } },
```

```
...    { $group: {
...        _id: "$user_attempted", failures: { $sum: 1 },
...        last_attempt: { $max: "$timestamp" }
...    }},
...    { $sort: { failures: -1 } }
... ])
```

```
[
  { _id: 'charlie', failures: 3, last_attempt: 17:00:05 },
  { _id: 'root',    failures: 2, last_attempt: 10:05:23 },
  { _id: 'admin',   failures: 1, last_attempt: 10:05:24 }
]
```

4.3 A3 — Répartition des sévérités

```
\prompt db.security_logs.aggregate([
...    { $group: { _id: "$severity", count: { $sum: 1 } } },
...    { $sort: { count: -1 } }
... ])
```

```
[ { _id: 'medium', count: 5 }, { _id: 'critical', count: 3 },
  { _id: 'high', count: 1 },   { _id: 'low', count: 1 },
  { _id: 'info', count: 1 } ]
```

4.4 A4 — Alertes par device

```
\prompt db.security_logs.aggregate([
...    { $match: { severity: { $in: ["high","critical"] } } },
...    { $group: {
...        _id: "$device_id", alert_count: { $sum: 1 },
...        event_types: { $addToSet: "$event_type" }
...    }},
...    { $sort: { alert_count: -1 } }
... ])
```

```
[
  { _id: 'endpoint-pc-042', alert_count: 1, event_types: ['
      malware_detected']          },
  { _id: 'siem-console',    alert_count: 1, event_types: ['
      privilege_escalation']      },
  { _id: 'firewall-main',   alert_count: 1, event_types: ['
      data_exfiltration_suspect'] },
  { _id: 'ids-sensor-02',   alert_count: 1, event_types: ['port_scan']
                        }
]
```

4.5 A5 — Événements par heure

```
\prompt db.security_logs.aggregate([
...   { $match: { timestamp: {
...       $gte: new Date("2025-03-03T00:00:00Z"),
...       $lt:  new Date("2025-03-04T00:00:00Z")
...   }}},
...   { $group: {
...       _id: { $dateToString: { format: "%H:00", date: "$timestamp"
   } },
...       count: { $sum: 1 }
...   }},
...   { $sort: { _id: 1 } }
... ])
```

```
[
  { _id: '10:00', count: 4 }, { _id: '11:00', count: 1 }, { _id:
     '14:00', count: 1 },
  { _id: '15:00', count: 1 }, { _id: '16:00', count: 1 },
  { _id: '17:00', count: 3 }
]
// 2 pics: 10h (reconnaissance) et 17h (exploitation)
```

Astuce

Placez toujours `$match` en premier. MongoDB peut alors utiliser les index, ce qui accélère considérablement les pipelines sur de gros volumes.

Chapitre 5

Index & explain()

Sans index, MongoDB parcourt chaque document (**COLLSCAN**, $O(n)$). Un index B-tree trie les valeurs : recherche en $O(\log n)$. Chaque index consomme de la mémoire et ralentit les écritures — ne pas d'index « au cas où » sur un SOC à 10 000 logs/seconde.

Composé : `{timestamp: 1, src_ip: 1}` accélère timestamp seul ou timestamp + src_ip, mais **PAS** src_ip seul (**prefix rule**).

5.1 Étape 1 — Sans index (COLLSCAN)

```
\prompt db.security_logs.find({ src_ip: "192.168.1.105" }).explain("
    executionStats")
```

```
{
  queryPlanner: { winningPlan: { stage: 'COLLSCAN' } },
  executionStats: {
    nReturned: 3, totalDocsExamined: 11,
    totalKeysExamined: 0, executionTimeMillis: 0
  }
}
```

5.2 Étape 2 & 3 — Index simple (IXSCAN)

```
\prompt db.security_logs.createIndex({ src_ip: 1 })
\prompt db.security_logs.find({ src_ip: "192.168.1.105" }).explain("
    executionStats")
```

```
{
  queryPlanner: {
    winningPlan: {
      stage: 'FETCH',
      inputStage: { stage: 'IXSCAN', indexName: 'src_ip_1' }
    }
  },
  executionStats: {
    nReturned: 3,
    totalDocsExamined: 3,     // était 11 avant
```

```
    totalKeysExamined: 3      // était 0 avant
  }
}
```

5.3 Étape 4 — Index composé

```
\prompt db.security_logs.createIndex({ timestamp: 1, src_ip: 1 })
\prompt db.security_logs.find({
...   timestamp: {
...     $gte: new Date("2025-03-03T10:00:00Z"),
...     $lte: new Date("2025-03-03T11:00:00Z")
...   },
...   src_ip: "192.168.1.105"
... }).explain("executionStats")
```

```
{
  queryPlanner: {
    winningPlan: {
      inputStage: {
        stage: 'IXSCAN',
        indexName: 'src_ip_1'  // Petit dataset -> MongoDB choisit
           src_ip_1
      }
    },
    rejectedPlans: [
      { inputStage: { indexName: 'timestamp_1_src_ip_1' } }
    ]
  }
}
// En production (millions de logs): timestamp_1_src_ip_1 serait
   choisi
```

5.4 Prefix rule — Piège

```
\prompt db.security_logs.find({ src_ip: "10.0.1.200" }).explain("
   executionStats")
```

```
{
  winningPlan: {
    inputStage: {
      stage: 'IXSCAN',
      indexName: 'src_ip_1'  // Utilise src_ip_1, PAS
         timestamp_1_src_ip_1
    }
  }
}
// Preuve prefix rule: {timestamp:1, src_ip:1} n'accélère PAS src_ip
   seul
```

Checkpoint 3 — Pipeline sans $match

Risque : `$group` traite TOUS les documents. Sur des millions de logs = consommation massive CPU/RAM → DoS applicatif involontaire.

Chapitre 6

Sécurité Data-Level

Minimisation (RGPD, art. 5.1.c) : ne stocker que le strictement nécessaire. **Sur-collecte** : un log avec User-Agent, MAC, contenu HTTP crée un risque inutile. **Masquage :** email → `"a***@soc.local"`.

La sur-collecte est une bombe à retardement

Si `security_logs` contient emails, FQDNs internes, tokens d'auth dans `raw_log`, une fuite expose l'infrastructure et l'identité des utilisateurs. **Chaque champ stocké est un champ à protéger.**

6.1 TP 6 — PII & Projections sûres

6.1.1 Masquer les emails

```
\prompt db.users.updateMany({}, [
...   { $set: { email_masked: {
...       $concat: [
...         { $substr: ["$email", 0, 1] }, "***@",
...         { $arrayElemAt: [{ $split: ["$email","@"] }, 1] }
...       ]
...   }}}
... ])
```

```
{ acknowledged: true, matchedCount: 4, modifiedCount: 4 }
```

```
\prompt db.users.find({}, { username: 1, email: 1, email_masked: 1,
   _id: 0 })
```

```
[
  { username: 'alice_analyst',  email: 'alice@soc.local',
     email_masked: 'a***@soc.local' },
  { username: 'bob_senior',     email: 'bob@soc.local',
     email_masked: 'b***@soc.local' },
  { username: 'charlie_admin',  email: 'charlie@soc.local',
     email_masked: 'c***@soc.local' },
  { username: 'diana_junior',   email: 'diana@soc.local', email_masked
     : 'd***@soc.local' }
```

```
]
```

6.1.2 Projection sûre (viewer) & Exclusion raw_log

```
\prompt db.users.find(
...   { is_active: true },
...   { username: 1, team: 1, roles: 1, email_masked: 1, _id: 0 }
... )
\prompt db.security_logs.find(
...   { severity: "critical" },
...   { raw_log: 0 }
... )
```

Q : {raw_log : 0} vs {src_ip : 1, timestamp : 1, _id : 0} ?

R : L'exclusion retourne tous les champs sauf celui-ci, y compris de futurs champs ajoutés par erreur. L'inclusion ne retourne QUE les champs listés → plus sûr.

6.2 Checklist Sécurité Data — SOC-lite

Périmètre fonctionnel : users (RBAC), security_logs (événements), incidents (enquêtes)

PII identifiés : email, full_name (users) ; raw_log, user_attempted (logs)

PII masqués : email → email_masked via aggregation update

Projections par rôle : viewer (inclusion), analyst (complet), admin (complet)

raw_log restreint aux analystes seniors (`{raw_log: 0}`)

Dénormalisation : username dupliqué (acceptable), pas de full_name

Index sûrs : timestamp+src_ip, src_ip (pas de user_attempted)

Pas de sur-collecte : pas de User-Agent, MAC, payload HTTP

3 principes sécurité data-level

1) **Minimisation** — ne stocker que le nécessaire. 2) **Masquage** — PII tronqués pour non-privilégiés. 3) **Projection par inclusion** — lister les champs autorisés plutôt qu'exclure les interdits.